

Introduction To NumPy: Arrays & Operations



Suman

5 Feb, 2026 At 8:00 PM

Notes

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Session 3.1: Introduction to NumPy: Arrays & Operations

In-Class Resources

Lecture Notes

Program: AIM101 — Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School

Mental Map

NumPy Fundamentals

- Why NumPy?
 - Speed (50-100x faster than lists)
 - Contiguous memory storage
 - Compiled C code operations
- Array Creation
 - `np.array()` — from lists
 - `np.zeros()`, `np.ones()`, `np.full()`
 - `np.arange()` — step-based sequences
 - `np.linspace()` — count-based sequences
 - `np.random.rand()`, `randint()`
- Array Attributes
 - `.shape` — dimensions
 - `.dtype` — data type
 - `.ndim` — number of dimensions
 - `.size` — total elements
- Element-wise Operations
 - Arithmetic: `+`, `-`, `*`, `/`, `**`
 - Comparison: `>`, `<`, `==`, `>=`, `<=`

- └─ List ≠ Array (CRITICAL!)
- └─ Broadcasting
 - └─ Scalar + Array
 - └─ 1D + 2D
 - └─ Shape compatibility rules
- └─ Vector Visualization
 - └─ matplotlib + NumPy

The Big Picture

Why NumPy Exists

Python lists are **flexible but slow** for numerical operations because:

- Each element is stored separately in memory (scattered)
- Type checking happens at runtime
- Operations require Python loops

NumPy arrays are **specialized and fast** because:

- Elements stored in contiguous memory blocks
- Single data type (no runtime type checking)
- Operations executed in compiled C code

The Result: NumPy is typically **50-100x faster** than equivalent Python list operations.

The Analogy

Python lists are like hand-writing calculations on paper—flexible but slow. **NumPy arrays** are like having a calculator—specialized and FAST.

Main Content

1. Importing NumPy

The standard convention:

```
import numpy as np
```

Everyone in the data science world uses `np` as the alias.

2. Array Creation Methods

From Python Lists

```
my_list = [10, 20, 30, 40, 50]
arr = np.array(my_list)
# array([10, 20, 30, 40, 50])
```

2D Arrays (Matrices)

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
# Shape: (2, 3) – 2 rows, 3 columns
```

Pre-filled Arrays

```
np.zeros(5)           # [0., 0., 0., 0., 0.]
np.zeros((2, 3))      # 2x3 array of zeros
np.ones((3, 4))       # 3x4 array of ones
np.full((2, 2), 99)   # 2x2 array filled with 99
```

Sequences

```
# arange: start, stop (EXCLUSIVE), step
np.arange(0, 10, 2)   # [0, 2, 4, 6, 8]

# linspace: start, stop (INCLUSIVE), number of points
np.linspace(0, 10, 5) # [0., 2.5, 5., 7.5, 10.]
```

Key Difference:

- `arange` uses a **step** and **excludes** the endpoint
- `linspace` uses a **count** and **includes** the endpoint

Random Arrays

```
np.random.rand(5)           # 5 floats in [0, 1)
np.random.randint(1, 100, size=5) # 5 integers in [1, 100)
np.random.randn(3)          # 3 values from standard normal
```

3. Array Attributes

Attribute	Description	Example
<code>.shape</code>	Dimensions as tuple	<code>(3, 4)</code> = 3 rows, 4 cols
<code>.ndim</code>	Number of dimensions	<code>2</code> for a matrix

Attribute	Description	Example
<code>.size</code>	Total element count	12 for a 3x4 matrix
<code>.dtype</code>	Data type	int64 , float64

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
arr.shape # (2, 3)
arr.ndim  # 2
arr.size   # 6
arr.dtype  # int64
```

Understanding Shape

- **1D:** (5,) — 5 elements, single dimension (note the comma!)
- **2D:** (3, 4) — 3 rows x 4 columns
- **3D:** (2, 3, 4) — 2 layers x 3 rows x 4 columns

Type Upcasting

When mixing types, NumPy converts to the type that preserves all information:

```
np.array([1, 2, 3.5]) # dtype: float64 (int → float)
```

4. Element-wise Operations

⚠ CRITICAL: List ≠ Array

Operation	Python List	NumPy Array
<code>+</code>	Concatenation	Element-wise addition
<code>*</code>	Repetition	Element-wise multiplication

```
# LIST behavior
[1, 2] + [3, 4] # [1, 2, 3, 4] - concatenation!
[1, 2] * 2      # [1, 2, 1, 2] - repetition!

# ARRAY behavior
np.array([1, 2]) + np.array([3, 4]) # [4, 6] - addition!
np.array([1, 2]) * 2                # [2, 4] - multiplication!
```

This is the #1 source of confusion for beginners!

Arithmetic Operations

```
arr = np.array([10, 20, 30, 40, 50])

arr + 5      # [15, 25, 35, 45, 55]
arr - 10     # [0, 10, 20, 30, 40]
arr * 2      # [20, 40, 60, 80, 100]
arr / 5      # [2., 4., 6., 8., 10.]
arr ** 2     # [100, 400, 900, 1600, 2500]
```

Array + Array

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([10, 20, 30])

arr1 + arr2  # [11, 22, 33]
arr1 * arr2  # [10, 40, 90]
```

Comparison Operations

Comparisons return **boolean arrays**:

```
scores = np.array([45, 67, 82, 55, 90])

scores > 60  # [False, True, True, False, True]
scores == 55 # [False, False, False, True, False]
```

5. Broadcasting

Broadcasting allows NumPy to work with arrays of different shapes.

Rule 1: Scalar Broadcasting

A scalar broadcasts to every element:

```
np.array([1, 2, 3]) + 10
# Conceptually: [1, 2, 3] + [10, 10, 10] = [11, 12, 13]
```

Rule 2: 1D + 2D Broadcasting

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
row = np.array([10, 20, 30])

matrix + row
# [[11, 22, 33],
#  [14, 25, 36]]
```

The 1D row broadcasts across each row of the matrix.

When Broadcasting Fails

Shapes must be compatible:

```
arr_3 = np.array([1, 2, 3])    # shape (3,)
arr_4 = np.array([1, 2, 3, 4]) # shape (4,)

arr_3 + arr_4 # ERROR! Incompatible shapes
```

6. Visualizing Vectors

Arrays with 2 elements represent 2D points/vectors:

```
import matplotlib.pyplot as plt

v1 = np.array([3, 2])
v2 = np.array([1, 4])
v_sum = v1 + v2 # [4, 6] - vector addition!

# Plot using quiver
plt.quiver(0, 0, v1[0], v1[1], color='blue', angles='xy', scale_units='xy', scale=1)
plt.quiver(0, 0, v2[0], v2[1], color='red', angles='xy', scale_units='xy', scale=1)
plt.quiver(0, 0, v_sum[0], v_sum[1], color='green', angles='xy', scale_units='xy', scale=1)
```

Key Frameworks

Speed Comparison Framework

```
# Always benchmark list vs array for numerical operations
python_list = list(range(1_000_000))
numpy_array = np.array(range(1_000_000))

%timeit [x * 2 for x in python_list] # ~50-100 ms
%timeit numpy_array * 2             # ~1-2 ms
```

Array Creation Decision Tree

```
What do you need?
|— Convert existing list? → np.array(my_list)
|— Start with zeros/ones? → np.zeros() / np.ones()
|— Fill with specific value? → np.full()
|— Sequence with step? → np.arange(start, stop, step)
|— Evenly spaced points? → np.linspace(start, stop, num)
|— Random values? → np.random.rand() / randint()
```

Debugging Framework

When something goes wrong:

Check shape: `arr.shape`

Check dtype: `arr.dtype`

Check size: `arr.size`

Print the array: `print(arr)`

Common Errors Quick Reference

Error	Likely Cause	Solution
<code>ValueError: operands could not be broadcast</code>	Shape mismatch	Check <code>.shape</code> of both arrays
Unexpected concatenation	Using <code>+</code> on lists, not arrays	Convert to NumPy array first
<code>TypeError: 'numpy.ndarray' object is not callable</code>	Using <code>()</code> instead of <code>[]</code>	Use <code>arr[0]</code> not <code>arr()</code>
Wrong dtype	Mixed types causing upcasting	Specify dtype explicitly

Key Takeaways

NumPy is 50-100x faster than Python lists for numerical operations

Array creation methods:

- `np.array()` — from lists
- `np.zeros()`, `np.ones()`, `np.full()` — pre-filled
- `np.arange()` — step-based (excludes endpoint)
- `np.linspace()` — count-based (includes endpoint)

Four key attributes: shape, dtype, ndim, size

Element-wise operations work on ALL elements without loops

CRITICAL: List behavior $\neq$ Array behavior with `+` and `*`

Broadcasting automatically stretches smaller arrays

Next Session Preview

Session 3.2: NumPy Power Tools: Indexing & Advanced Operations

- Fancy indexing (selecting non-contiguous elements)
- Boolean masking (filtering with conditions)
- Reshaping arrays
- Universal functions (ufuncs)
- Views vs copies

Mini-Project: Student Grade Analyzer

Vishlesan i-Hub IIT Patna × Masai School